

CSC490: Assignment 1

Team Members

Name	Student ID
Aarya Prakash	1008790257
Derek Huynh	1008849052
Merrick Liu	1008951920
Rhys Balevicius	1000801974

University of Toronto
Department of Computer Science

1 Interest Statements

“Cursor just doesn’t work well. Like it doesn’t know how to code LinkedIn code.”
– Software Engineer at LinkedIn

AI coding agents are powerful but often fail in real-world environments due to a lack of relevant context. Currently, most agents are restricted to the local working directory, causing them to miss valuable context like external dependencies, other services, and historical design decisions. We aim to solve this by building a system that surfaces non-local context. We care about this problem because as software engineers, we have seen how this “context gap” leads to failures, high costs, and technical risks. We believe that providing agents with a more comprehensive view of the codebase is key to unlocking their full potential.

- **Aarya:** I am excited to develop our systems context retrieval/selection strategy along with how we ingest data because in my persona experience, better context reduces the prompt-review-debug loop
- **Derek:** The area I want to contribute to most is improving the performance and efficiency of our system because in production environments, our engine will be processing 100k+ of queries concurrently.
- **Merrick:** I am most excited about implementing the search engine and building an evaluation system, since there are many ways to improve retrieval (e.g., reranking, vector databases, agentic elements) and evaluations will let us measure each iteration’s impact and benchmark against existing solutions.
- **Rhys:** I am excited to develop rigorous evaluation metrics that guide research into formal methods and measure improvements to how agents reason about long-horizon tasks with our tool.

2 Landscape Analysis (1/2)

We identified each of these companies as they all have some advanced context system or are likely to build one. They range from established companies to early stage start ups.

Relevant Item	Description	Commentary
Sourcegraph	Enterprise code intelligence platform that provides services like code search tools (Deep Search and Code Search), analytics, and a coding agent. They provide a web interface to their tools, but also a MCP server .	Their search system only indexes Github repositories which leaves a lot of valuable context on the table such as internal docs outside of the repositories. Their system to bring in context from sources like Jira is very manual.
Nia	A platform that surfaces relevant and up to date context to AI agents focused on reducing hallucinations. It is exposed as an API and MCP. It ingests data from external and internal knowledge bases.	Early stage startup doing something very similar to us. Pivoted from context for coding agents to general agents. This generality could be a weakness since their are a lot of domain specific patterns for “coding context”.
AugmentCode	An AI coding agent that utilizes their “Context Engine” which ingests various data sources and surfaces relevant context. The engine is also exposed as an MCP	Cross repository context is restricted to local directories which does not scale. Furthermore, even though their Context Engine SDK allows for arbitrary indexing, you still have to build your own scripts around it.
Qodo	Primarily an AI-powered code review platform, but also has a “Context Engine” that is essentially RAG over code repositories. It is exposed as a MCP, API, and chatbot.	Only indexes repositories so loses context in other sources like internal documentation in Notion, etc.
Tabnine	AI coding assistant focused on tab completion and integration into other IDEs. Developers are able to index other repositories outside of their local development environment through “Connection” .	Their context provider is not exposed as a stand alone product so developers who prefer other coding agents are unable to use this. In other words, you cannot drop it in into other tools.

Landscape Analysis (2/2)

Relevant Item	Description	Commentary
Grep App (Vercel)	Code search engine across millions of public GitHub repositories that is known for its speed. It also has an MCP server	Only indexes public code repositories so not useful when task requires internal/private context ie. internal microservices.
GitHub	A cloud-based platform for code storage, management, etc. They have a Code Search product that is also exposed as part of their MCP server .	Restricted only to GitHub repositories meaning context from internal documentation outside of GitHub is unavailable.
Glean	An enterprise search engine that ingests data from various sources like Slack, Google Drive, Jira, etc. through their connectors . They have a Github connector and an MCP server .	Potentially one of our largest competitors. From personal experiences and conversations with users, we have heard mixed reviews. Despite having various connectors, people do not seem to be using it to enhance their coding agents.
Algolia	“Search-as-a-Service” platform. It can index arbitrary data from websites to code. Traditionally used by companies to build internal/external search features, but also has an MCP server .	Their disadvantage is their generality. Yes, you can technically index code, but their system is not optimized for that compared to solutions specifically built for code. There are structures like dependencies in code that Algolia can’t capture.
Devin (Cognition)	An “AI software engineer” that runs in the background autonomously. It operates in its own VM essentially giving it access to a computer.	Its integrations with external sources like Linear are not designed to pull data. Cloning multiple repositories for context into its VM is not scalable.
Other Coding Agent Tools	Products like Claude Code , Codex , Gemini CLI , OpenCode , Warp , etc. Most are terminal based coding agents, but some have started to transitioned to have web clients too.	These agents are scoped to the directory they are opened in and their only way to pull context from outside their local scope is through a web search tool if they have one. However, we anticipate that they will move past this “root directory” restriction soon.

3 Project Outline

3.1 Problem Statement

Although AI coding agents excel at completing self-contained tasks, they consistently struggle within complex large-scale engineering environments. This performance gap is driven by a lack of context such as external dependencies, architectural constraints, and existing services. As a result, their utility remains limited to isolated snippets rather than end-to-end development.

3.2 Proposed Solution

Our solution is a codebase search engine that indexes data from GitHub, internal documentation, and other relevant technical sources. It will be exposed as a Model Context Protocol service ([MCP](#)), allowing it to be plugged into any AI coding agent so it can retrieve specific relevant information needed for its task.

3.3 Technical Approach

Our system consists of three core components: ingestion, storage, and retrieval. For ingestion, we will leverage prebuilt connectors, our own custom integrations, and an appropriate embedding model. For storage, we will combine vector, graph, and relational databases. For retrieval, we will evaluate multiple strategies: hybrid search, graph-based retrieval, agentic search workflows, etc.

Milestone	Description
Ingestion / Storage	Build data ingestion pipeline along with storage system.
Search Engine V1	Initial prototype and evaluation of search tool involving hybrid retrieval, code-aware chunking, and reranking.
MCP service scaffold	Implement a MCP server with basic tool endpoint(s) and integrate it with Cursor and Claude Code.
Evaluation system	Define performance metrics, construct benchmark dataset(s), and perform a baseline evaluation with existing tools.
Iterative R&D Loops	Multiple experiments/iterations with how we can improve our search engine through various techniques/adjustments.

3.4 Key Unknowns to Investigate

- Why do people not use some of the products listed in the landscape?
- How comfortable do companies feel purchasing a “plugin” rather than a full agent?
- What is considered and how is “relevant” context determined?
- What are the best strategies for how we structure and retrieve data?
- How will we keep the data in sync and our search engine fresh?

4 Project Press Release

Introducing Lighthouse

A Unified Context Layer for AI Coding Across the Enterprise

Toronto, Canada — [Future Date]

Today, our team announced **Lighthouse**, a centralized context and code intelligence platform designed to make AI coding agents effective at the scale of real organizations. Lighthouse serves as a single source of truth for coding model context, unifying codebases, documentation, and institutional knowledge across an organization's many repositories.

As AI coding agents become embedded in daily development workflows, teams are discovering a critical limitation. While agents can generate correct code in isolation they miss implicit design decisions, non-obvious dependencies, and existing services. The result is wasted time, broken assumptions, and growing skepticism from engineers.

Modern coding agents are fast, but they are often blind to the organizational reality they operate in. They see fragments, not systems.

— Lighthouse project team

Lighthouse addresses this problem by acting as an organization-wide context layer for coding models. It continuously ingests and indexes source code, tests, architectural relationships, historical changes, and domain knowledge across disparate repositories. Exposed as a Model Context Protocol service, it is compatible with every coding agent.

Instead of hoping the model stumbles onto the right files, Lighthouse makes context explicit, searchable, and grounded in how the system actually works.

— Project contributor

Engineering leaders see Lighthouse as an organizational productivity unlock rather than a developer convenience. By unifying context at the organizational level, Lighthouse improves coding model efficiency across teams. Engineers spend less time correcting agents, reviewing misguided changes, and re-explaining system assumptions.

This is not just about better code suggestions. It is about embedding tribal knowledge into the development process so every engineer and every agent can benefit from it.

— VP of Engineering

Lighthouse is currently in active development, with upcoming milestones focused on advanced code-aware search, richer organizational knowledge ingestion, and rigorous measurement of agent efficiency and correctness in real production environments.

Appendix

A Project Press Release Iterations

A.1 Iteration 1: Reliability-First Framing

Introducing Lighthouse

Making AI Coding Agents Trustworthy in Real Software Systems

Toronto, Canada — [Future Date]

Today, our team announced **Lighthouse**, a new reliability layer for AI coding agents that helps them make safer, more informed code changes in real-world software systems. Lighthouse integrates directly with existing AI coding tools and editors, improving how agents retrieve context, reason about changes, and validate edits before they introduce regressions.

AI coding agents have quickly become part of everyday development workflows. However, while these tools excel at producing syntactically correct code, they often fail in realistic environments: large codebases, implicit design decisions, incomplete documentation, and fragile test suites. These failures increase engineering risk, slow teams down, and erode trust in AI-assisted development.

Today’s agents don’t fail because they can’t write code. They fail because they don’t know what to look at, why something exists, or how to confirm that a change is safe.

— Project team member

Lighthouse addresses this gap by acting as a context and validation substrate for coding agents. Built as a Model Context Protocol (MCP) service, it ingests and indexes engineering artifacts such as source code, tests, dependency graphs, and historical signals. When an agent attempts a change, Lighthouse helps it retrieve the right context and apply validation checks that catch regressions early.

For the first time, search and validation become explicit system components instead of hidden model behavior. That’s how human engineers work—and that’s the behavior we want to operationalize.

— Project contributor

Early users report fewer broken builds, faster reviews, and greater confidence in AI-generated changes.

Before Lighthouse, I spent more time reviewing AI code than writing it. Now the agent comes in better informed, and the changes make sense in the broader system.

— Senior software engineer (early testing)

Engineering leaders also see the potential impact.

This kind of tooling is essential if AI agents are going to scale beyond toy tasks. Reliability is the difference between experimentation and production.

— VP of Engineering

The team is also investing heavily in measurement, building benchmarks that reflect real engineering failures—non-local breakages, duplicated logic, and violations of implicit invariants—rather than superficial metrics like pass rates on small tasks.

Our goal isn't to make agents look smarter. It's to make them safer and more predictable in the workflows teams actually rely on.

— Project team

Lighthouse is currently in active development, with upcoming milestones focused on advanced code-aware search, validation tooling, and rigorous evaluation of reliability improvements.

A.2 Iteration 2: Identifying Performance and Efficiency Pain Points

Introducing Lighthouse

A Unified Context Layer for AI Coding Across the Enterprise

Toronto, Canada — [Future Date]

Today, our team announced **Lighthouse**, a centralized context and code intelligence platform designed to make AI coding agents effective and trustworthy at the scale of real organizations. Lighthouse serves as a single source of truth for coding model context, unifying codebases, documentation, and institutional knowledge across an organization's many repositories.

As AI coding agents become embedded in daily development workflows, teams are discovering a critical limitation. While agents can generate correct code in isolation, they often operate with incomplete or incorrect context. They miss implicit design decisions, overlook non-obvious dependencies, and surface irrelevant files while ignoring the code that actually matters. The result is wasted time, broken assumptions, and growing skepticism from engineers.

Modern coding agents are fast, but they are often blind to the organizational reality they operate in. They see fragments, not systems.

— Lighthouse project team

Lighthouse addresses this problem by acting as an organization-wide context layer for coding models. Built as a Model Context Protocol service, it continuously ingests and indexes source code, tests, architectural relationships, historical changes, and domain knowledge across disparate repositories. When an agent is asked to make a change, Lighthouse ensures it is reasoning with the full, correct context of the organization, not a narrow or misleading slice.

Instead of hoping the model stumbles onto the right files, Lighthouse makes context explicit, searchable, and grounded in how the system actually works.

— Project contributor

Early users report a step change in confidence when working with AI-assisted development.

Before Lighthouse, I never knew what the agent didn't know. Now it feels like it is operating with the same background understanding a teammate would have.

— Senior software engineer, early testing

Engineering leaders see Lighthouse as an organizational productivity unlock rather than a developer convenience.

This is not just about better code suggestions. It is about embedding tribal knowledge into the development process so every engineer and every agent can benefit from it.

— VP of Engineering

By unifying context at the organizational level, Lighthouse improves coding model efficiency across teams. Engineers spend less time correcting agents, reviewing misguided changes, and re-explaining system assumptions. Agents make fewer incorrect decisions because they are grounded in complete and relevant knowledge.

When agents have the right context, trust follows. That is the foundation required for AI to meaningfully scale inside engineering organizations.

— Lighthouse team

Lighthouse is currently in active development, with upcoming milestones focused on advanced code-aware search, richer organizational knowledge ingestion, and rigorous measurement of agent efficiency and correctness in real production environments.